access even though the majority of functions for manipulating the data structure may be called from multiple threads concurrently without problem.

The second aspect to consider is that of enabling genuine concurrent access. I can't offer much in the way of guidelines here; instead, here's a list of questions to ask yourself as the data structure designer:

- Can the scope of locks be restricted to allow some parts of an operation to be performed outside the lock?
- Can different parts of the data structure be protected with different mutexes?
- Do all operations require the same level of protection?
- Can a simple change to the data structure improve the opportunities for concurrency without affecting the operational semantics?

All these questions are guided by a single idea: how can you minimize the amount of serialization that must occur and enable the greatest amount of true concurrency? It's not uncommon for data structures to allow concurrent access from multiple threads that merely read the data structure, whereas a thread that can modify the data structure must have exclusive access. This is supported by using constructs like `boost::shared_mutex`. Likewise, as you'll see shortly, it's quite common for a data structure to support concurrent access from threads performing different operations while serializing threads that try to perform the same operation.

The simplest thread-safe data structures typically use mutexes and locks to protect the data. Although there are issues with this, as you saw in chapter 3, it's relatively easy to ensure that only one thread is accessing the data structure at a time. To ease you into the design of thread-safe data structures, we'll stick to looking at such lock-based data structures in this chapter and leave the design of concurrent data structures without locks for chapter 7.

6.2. Lock-based concurrent data structures